Volume 08 | End-to-End Execution Trace

Connect offline build, application startup, browser request, and response.

Timeline A: offline build

1. Cleaning pipeline prepares station metadata and operational inputs.

2. Synthetic generator creates 19 slots/day across station roster and date range, derives scenario target from encoded patterns and saves large CSV plus typology and line-split JSON.

3. Training pipeline derives feature columns, makes chronological split at 2026-10-01, builds training-only history features, trains baselines and HistGradientBoostingRegressor, computes metrics and writes joblib artifacts/metrics JSON.

4. These are offline commands. The web request does not run them.

Timeline B: service startup

1. Procfile asks Uvicorn to import `run:app`.

2. `run.py` imports FastAPI application from package.

3. `app/__init__.py` creates FastAPI object, mounts `/static`, imports routes.

4. `routes.py` reads station CSV, builds station list, calls `build_graph`.

5. Import of forecast loads model and metadata; scheduling loads operations config; templates are pointed at `app/templates`.

6. Uvicorn begins accepting requests only if required import-time dependencies/files succeed.

Timeline C: future-date 09:00 JSON scenario

1. Browser serializes station/date/time/line and POSTs `/api/predict`.

2. FastAPI reads JSON; malformed JSON/non-object receives 400.

3. Async endpoint delegates synchronous `_predict_data` to threadpool.

4. Helper validates `YYYY-MM-DD` and `HH:MM`, gets date context, tests service window.

5. Forecast module gets station metadata, creates 23 feature values for each hourly point, selects exact saved column order, predicts and clips nonnegative.

6. It computes interpolated selected minute and full 19-point daily curve.

7. Scheduler distributes interchange demand by configured weights, calculates clearance, selected frequency/headway and risk under constraints.

8. Graph search from each source line-state finds minimum weighted times, maps line states to physical destinations, removes origin.

9. JSON response includes explicit data-provenance fields. JS ignores stale response if a newer request ID exists, then updates cards/charts/table.

Timeline D: closed-time request (e.g. 02:02)

Input parsing succeeds, calendar can still be populated, service gate returns false. Demand values are zero, and scheduling headline is replaced with `SERVICE_CLOSED`, zero trains, no headway, no capacity utilization. Graph travel-time estimates are still computed because network reachability is not dependent on whether trains are operating at that selected moment. UI should make closed service explicit; zero here means no service modeled, not zero recorded passengers.

Mapping worked example

Suppose route list labels `Rajiv Chowk` for Blue and Yellow Lines. Normalization maps each to canonical CSV label. Graph adds `(Rajiv Chowk, Blue Line)` and `(Rajiv Chowk, Yellow Line)` and adjacent station edges for each line. The interchange dictionary adds a 5-minute transfer edge. Search can reach a destination on either line; if path changes line mid-trip, it pays a transfer penalty. Repeated coordinates or implausible >8-minute adjacent distance use 2.5-minute fallback. Destination is output at minimum among its line nodes.

Debugging sequence

- Server won't start: inspect first import-time exception and verify model/CSV/template paths.
- API 400: inspect input JSON shape/date/time.
- Wrong station output: validate station canonical name and metadata index mapping.
- Unexpected low crowding: inspect synthetic demand and clearance denominator/assumptions.
- Missing destination: compare canonical mapping, line segments and interchange edges.
- Stale chart: inspect response schema, JS console and request IDs.
- Claims mismatch: source of truth is current data/model/config; update resume/docs and keep generated-vs-observed distinction.